MANCHESTER 1824
The University of Manchester

[redacted]

A dissertation submitted to The University of Manchester for the degree of

Master of Science in Advanced Computer Science

in the Faculty of Science and Engineering

Year of submission

2025

[redacted]

School of Engineering

Contents

Word count: 8789

Abstract

This project investigates a state-of-the-art digital watermarking framework designed for style protection in LLM-generated text, without relying on the conventional “green lists” mechanism. The method operates in three stages: Comparative Learning, Condensed Lists Building, and Watermark Extraction. In the first stage, sentence embeddings, pseudo-labels, and contrastive learning are used to construct positive and negative samples. In the second stage, LLMs generate stylistic explanations that are encoded into weighted style matrices. In the final stage, a learnable watermark matrix maps these features into binary anchors, enabling effective style-infringement detection. Extensive experiments—including few-shot evaluation, robustness, and interpretability analyses—demonstrate high technical accuracy, adaptability across diverse stylistic dimensions, and strong resistance to adversarial attacks. Ablation studies confirm the necessity of each component. While the method achieves robust sentence-level detection with few positive samples, challenges remain in hyperparameter sensitivity, prompt template design, LLM API limitations, paragraph-level protection, and multi-style watermarking. Future work may focus on principled hyperparameter selection, automatic prompt engineering, offline deployment strategies, and extension to paragraph- or document-level, multi-style protection. Overall, this project assesses a reliable, interpretable, and scalable approach for safeguarding human writing styles in LLM-generated content, providing a foundation for further advances in digital watermarking and style protection.

Declaration of originality

I hereby declare that this dissertation is my own original work, except where reference is made to the work of others, which has been clearly acknowledged.

Furthermore, I confirm that no portion of the work referred to in this dissertation has been submitted in support of an application for another degree or qualification at this or any other university or institute of learning.

Copyright statement

i The author of this dissertation (including any appendices and/or schedules to this dissertation) owns certain copyright or related rights in it (the “Copyright”) and s/he has given The University of Manchester certain rights to use such Copyright, including for administrative purposes.

ii Copies of this dissertation, either in full or in extracts and whether in hard or electronic copy, may be made *only* in accordance with the Copyright, Designs and Patents Act 1988 (as amended) and regulations issued under it or, where appropriate, in accordance with licensing agreements which the University has entered into. This page must form part of any such copies made.

iii The ownership of certain Copyright, patents, designs, trademarks and other intellectual property (the “Intellectual Property”) and any reproductions of copyright works in the thesis, for example graphs and tables (“Reproductions”), which may be described in this dissertation, may not be owned by the author and may be owned by third parties. Such Intellectual Property and Reproductions cannot and must not be made available for use without the prior written permission of the owner(s) of the relevant Intellectual Property and/or Reproductions.

iv Further information on the conditions under which disclosure, publication and commercialisation of this dissertation, the Copyright and any Intellectual Property and/or Reproductions described in it may take place is available in the University IP Policy (see `http://documents.manchester.ac.uk/display.aspx?DocID=24420`), in any relevant Dissertation restriction declarations deposited in the University Library, The University Library’s regulations (see `http://www.library.manchester.ac.uk/about/regulations/`) and in The University’s policy on Presentation of Theses.

Acknowledgments

I would like to express my sincere gratitude to [redacted] for his invaluable guidance and insightful feedback throughout the process of writing this thesis. His expertise and encouragement have been instrumental in shaping the quality of this work.

I am also deeply grateful to my family and friends for their constant support, understanding, and encouragement. Their companionship and care have been a source of strength during the entire journey.

1 Introduction

1.1 Background

Since the release of ChatGPT-3 (Brown et al. 2020), large language models (LLMs) have undergone rapid and significant development. Modern LLMs such as ChatGPT-5 (OpenAI 2025), Grok (XAI 2024), and DeepSeek (DeepSeek 2024) have demonstrated remarkable capabilities in natural language generation, producing human-like text that is often indistinguishable from that written by actual humans (Lu et al. 2024a). As a result, LLMs have attracted significant attention from both industry and academia. However, these advancements have also raised concerns. Because LLMs require extensive textual data for training, many fear that copyrighted materials may be used without permission, potentially leading to their unauthorized reproduction during text generation (Bergman et al. 2022; Mirsky et al. 2023). Additionally, the misuse of LLMs for generating disinformation or fabricated news can have severe social consequences (Megías et al. 2021). For example, *The New York Times* recently filed a lawsuit against OpenAI, alleging unauthorized use of its copyrighted content (Michael Grynbaum 2023), along with other notable cases (Milmo 2023; Masons 2024). Therefore, it is of great importance to detect whether adversaries misuse human-written texts (such as novels or news articles) through large language models. Such detection concerns not only the misuse of textual content but also the misuse of textual style. Existing studies have primarily focused on identifying content misuse, while research on style misuse remains relatively limited (Z. Zhang et al. 2025). Consequently, developing effective methods to detect the misuse of human writing styles by large language models has become an urgent challenge, and digital watermarking has emerged as a promising approach to achieve both traceability and style preservation.

1.2 Related Work

Digital watermarking technology has recently attracted significant research interest due to its potential to detect content generated by large language models (LLMs) (Rastogi and Pruthi 2024). The core idea of using digital watermarking technology to detect text generated by large language models is as follows.

Large language models (LLMs) generate tokens in a step-by-step manner, where each new token is selected based on the given prompt and the tokens already produced. At each step, the model assigns probabilities to all possible tokens, and a sampling strategy is then applied to determine the next token. Common strategies include *top-k sampling* (Holtzman, Buys, Forbes, et al. 2018), which restricts choices to the k most likely tokens, and *top-p* (nucleus) sampling (Holtzman, Buys, Du, et al. 2019), which selects from the smallest set of tokens whose cumulative probability mass exceeds a given threshold.

Watermarking methods build on this process by subtly influencing the model's token selection in order to embed hidden statistical patterns. These patterns are invisible to human readers but they can later be detected by specialized algorithms. A detector computes a score for the given text, and if the score exceeds a pre-defined threshold, the text is classified as watermarked and therefore likely generated by an LLM.

The most popular existing approach for using digital watermarking technology to detect text generated by large language models relies on the "green lists" watermarking method. The idea of the "green list" watermarking method is to modify the assigned probabilities of specific tokens. Its detailed working process is as follows.

The watermarking process introduces a statistical bias into the model's token generation. At each position t, the vocabulary V is split into two groups: a *green list* G_t, which contains a fraction γ of the tokens, and a *red list* with the remaining tokens. A fixed bias δ is then added to the logits of green tokens, while other tokens remain unchanged. The resulting probability distribution is defined as:

$$\widetilde{P}_t(v) = \frac{\exp(l_t(v) + \delta \cdot \mathbf{1}\{v \in G_t\})}{\sum_{v' \in V} \exp(l_t(v') + \delta \cdot \mathbf{1}\{v' \in G_t\})}. \tag{1}$$

Here, $l_t(v)$ denotes the original logit value of token v at position t before applying the watermark bias, and $\mathbf{1}\{\cdot\}$ denotes an indicator function that equals 1 if the token v is in G_t and 0 otherwise. This mechanism slightly increases the likelihood of sampling green tokens, without altering the overall fluency of the generated text.

Detection is based on whether green tokens appear more frequently than expected. Given a sequence $(v_1, \ldots, v_T)$ and the green lists $\{G_t\}_{t=1}^T$, the detection score is computed as:

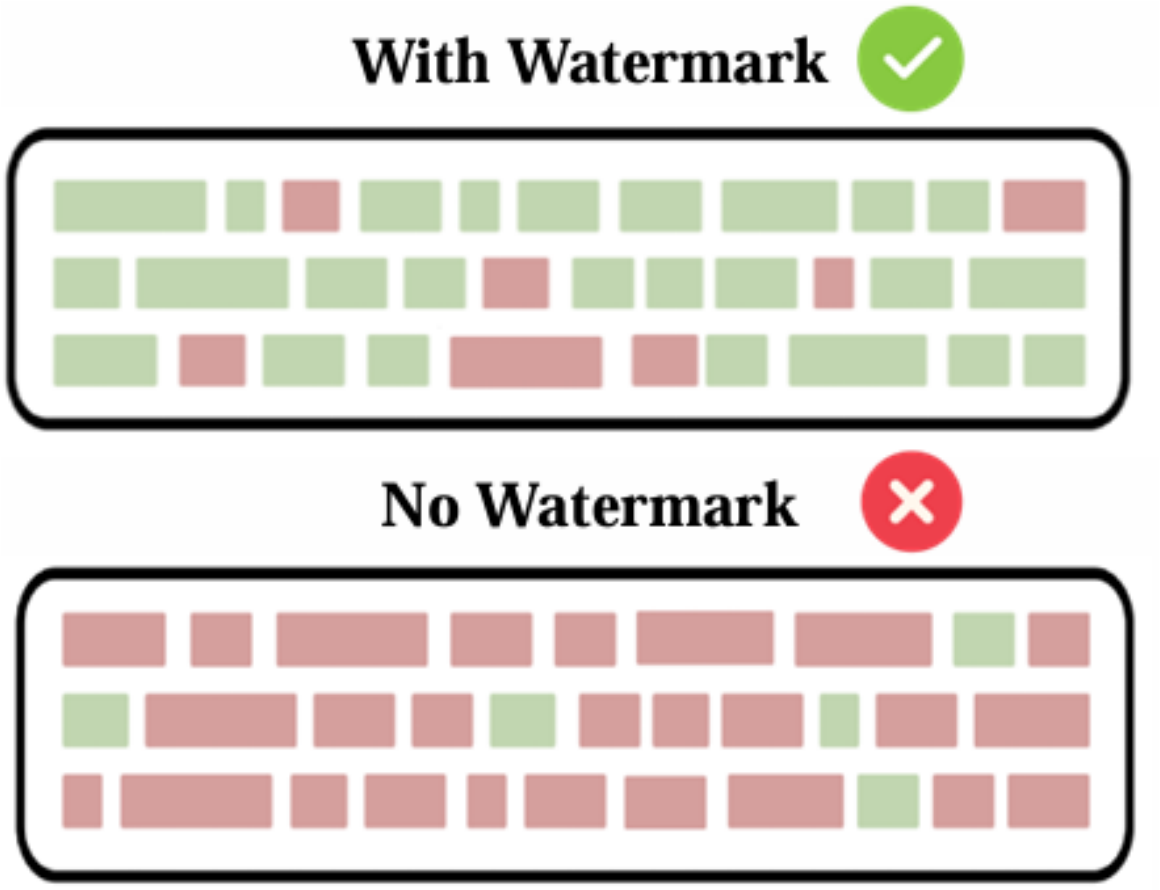


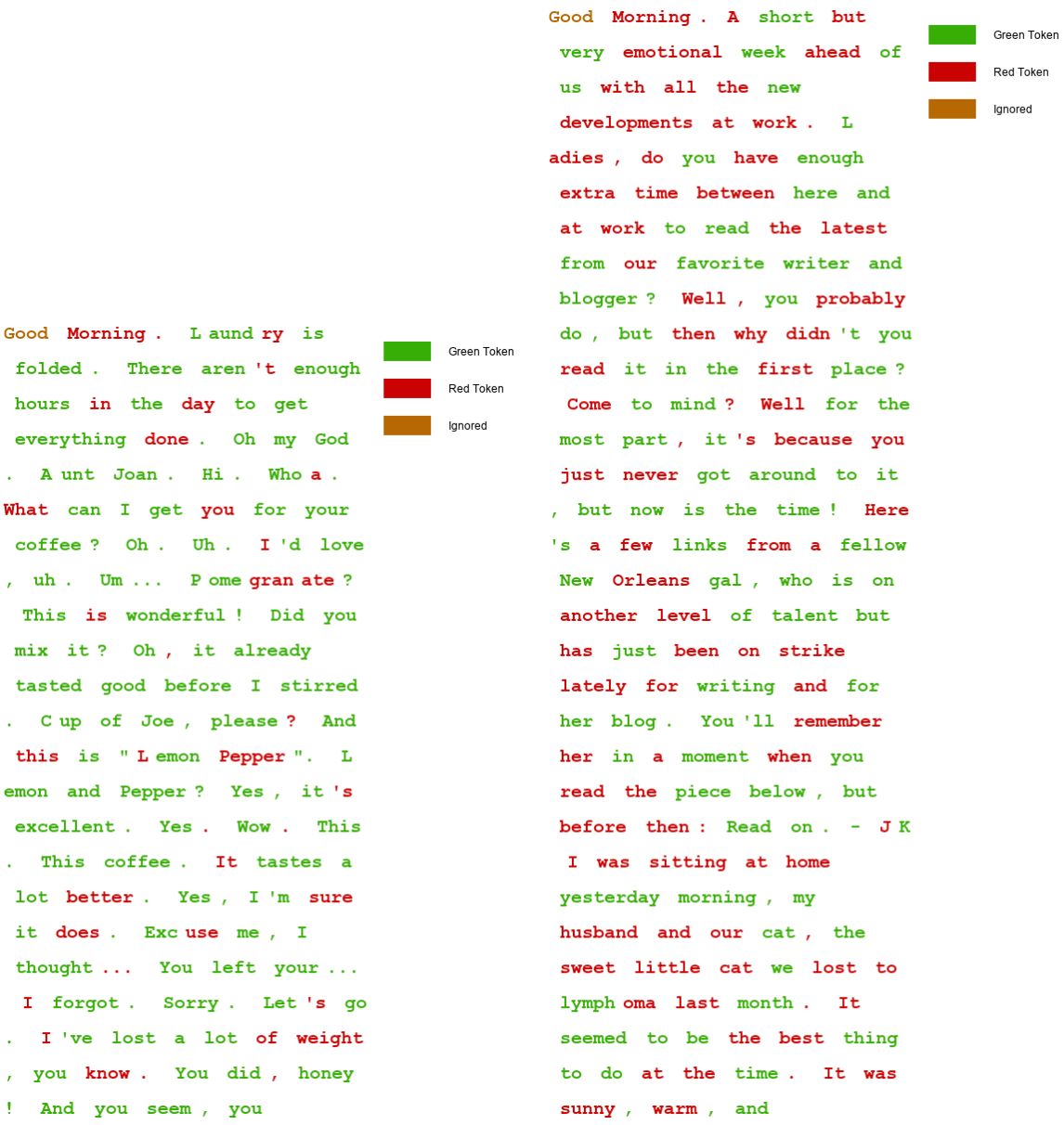
Fig. 1. The difference in the proportion of green tokens between watermarked text and unwatermarked text. (Pan et al. 2024)

$$d(v_1, \ldots, v_T) = \frac{\sum_{t=1}^{T} \left(\mathbf{1}\{v_t \in G_t\} - \gamma\right)}{T\gamma(1-\gamma)}. \tag{2}$$

If this score exceeds a threshold τ, the text is classified as watermarked. The parameter γ reflects the expected fraction of green tokens for non-watermarked text, while the denominator normalizes the score, reducing the chance of false positives, particularly when T is large.

As shown in Figure 1, watermarked text contains a much higher proportion of green tokens, whereas unwatermarked text exhibits a much smaller proportion.

Among the pioneering works in “green lists” watermarking methods is the KGW algorithm (Kirchenbauer et al. 2023), where Kirchenbauer et al. propose injecting imperceptible perturbations into training corpora as a watermarking strategy. Specifically, they divide the model’s vocabulary into two sets—green and red—and softly increase the probability of generating green tokens. This approach enables efficient post-hoc watermark detection while maintaining high text quality, and has since inspired numerous follow-up methods



(a) watermarked texts **(b)** unwatermarked texts

Fig. 2. A pair of texts, one watermarked and one unwatermarked, generated by the KGW algorithm.

that build on the ”green lists” concept. Figure 2 presents a visual example of the KGW algorithm, illustrating the difference in the proportion of green tokens between watermarked and unwatermarked text. As shown, the watermarked text contains a substantially higher proportion of green tokens, whereas the unwatermarked text exhibits only a minimal presence of green tokens.

Building on the foundation of the “green lists” approach, later works have proposed several improvements to enhance its effectiveness and adapt it to different text generation scenarios. For instance, Lu et al.’s EWD algorithm (Lu et al. 2024b) introduces entropy-based weighting, which gives higher importance to green tokens with greater entropy. This ad-

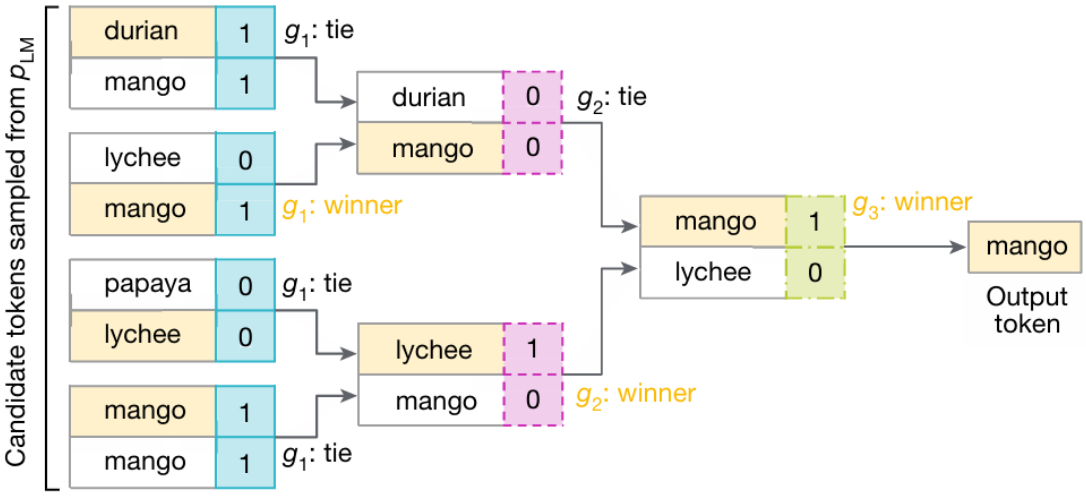


Fig. 3. The tournament sampling process used in SynthID-Text. (Dathathri et al. 2024)

justment enhances detection accuracy in low-entropy settings while maintaining robust performance in high-entropy scenarios, effectively broadening the applicability of the original method. Similarly, Zhao et al. present the UNIGRAM algorithm (Zhao et al. 2023), which employs a random key to divide tokens and increases the logits of green tokens using a strength parameter. This strategy improves resilience to editing attacks while maintaining the naturalness and variability of the generated text. More recently, Dathathri et al (Dathathri et al. 2024) proposed a tournament-based sampling method, SynthID-Text, to improve watermark detection, even when the text exhibits varying levels of randomness. In this approach, each token is assigned multiple scores using independent watermark functions, which depend on a secret key and the recent context.

The next token is then chosen through a series of mini-tournaments: several tokens are sampled and compete in pairs, with higher-scoring tokens advancing to the next round. This process is repeated multiple times until a final winner is selected as the output token. Figure 3 illustrates this procedure.

Since tokens with higher scores are more likely to be selected, the resulting text contains hidden patterns that can later be detected. If a dedicated detector finds that these patterns occur above a certain threshold, the text is classified as watermarked.

1.3 Motivation and Project objectives

Although the previously discussed ”green list”-based digital watermarking approaches have shown strong performance in detecting text produced by large language models (LLMs), they suffer from several inherent limitations.

First, to strengthen the watermark signal, these methods often increase the sampling probability of green-listed tokens, which inevitably alters the content of the watermarked text, as illustrated in Figure 4. This modification, while enhancing watermark detectability, may degrade the naturalness and overall quality of the generated text. As Chang et al. point out: ”This finding, in turn, reveals the crucial limitation of existing watermarking schemes: high watermark detectability and high text quality cannot be achieved at the same time, since the very same set of tokens causes quality degradation while contributing to watermark detectability simultaneously.” (Chang, Hassani, and Shokri 2024)

Secondly, this watermark detection paradigm is particularly vulnerable to a class of attacks known as *Watermark Smoothing Attacks*, in which adversaries manipulate the token distribution to reduce the prominence of green tokens, thereby evading detection. To the best of our knowledge, there is currently no effective defense mechanism specifically designed to counter such attacks. This highlights a significant limitation of existing watermarking approaches: while they can achieve reasonable detectability under normal conditions, their robustness against sophisticated adversarial manipulations remains largely unaddressed, exposing potential vulnerabilities in scenarios where model outputs might be deliberately altered to conceal watermarks.

Lastly, since human-written text can occasionally contain a high proportion of green-listed tokens, these approaches are prone to false positives, undermining detection accuracy and reliability. Such challenges hinder the widespread adoption of green lists-based watermarking in complex, real-world scenarios.

Motivated by these challenges, this project seeks to assess a state-of-the-art digital watermarking algorithm that does not depend on the “green lists” mechanism, thereby addressing the aforementioned limitations, with some modern text datasets serving as the source of protected text. Specifically, this project will systematically analyze the algorithm’s strengths and limitations. In addition to basic watermark detection, the project will conduct

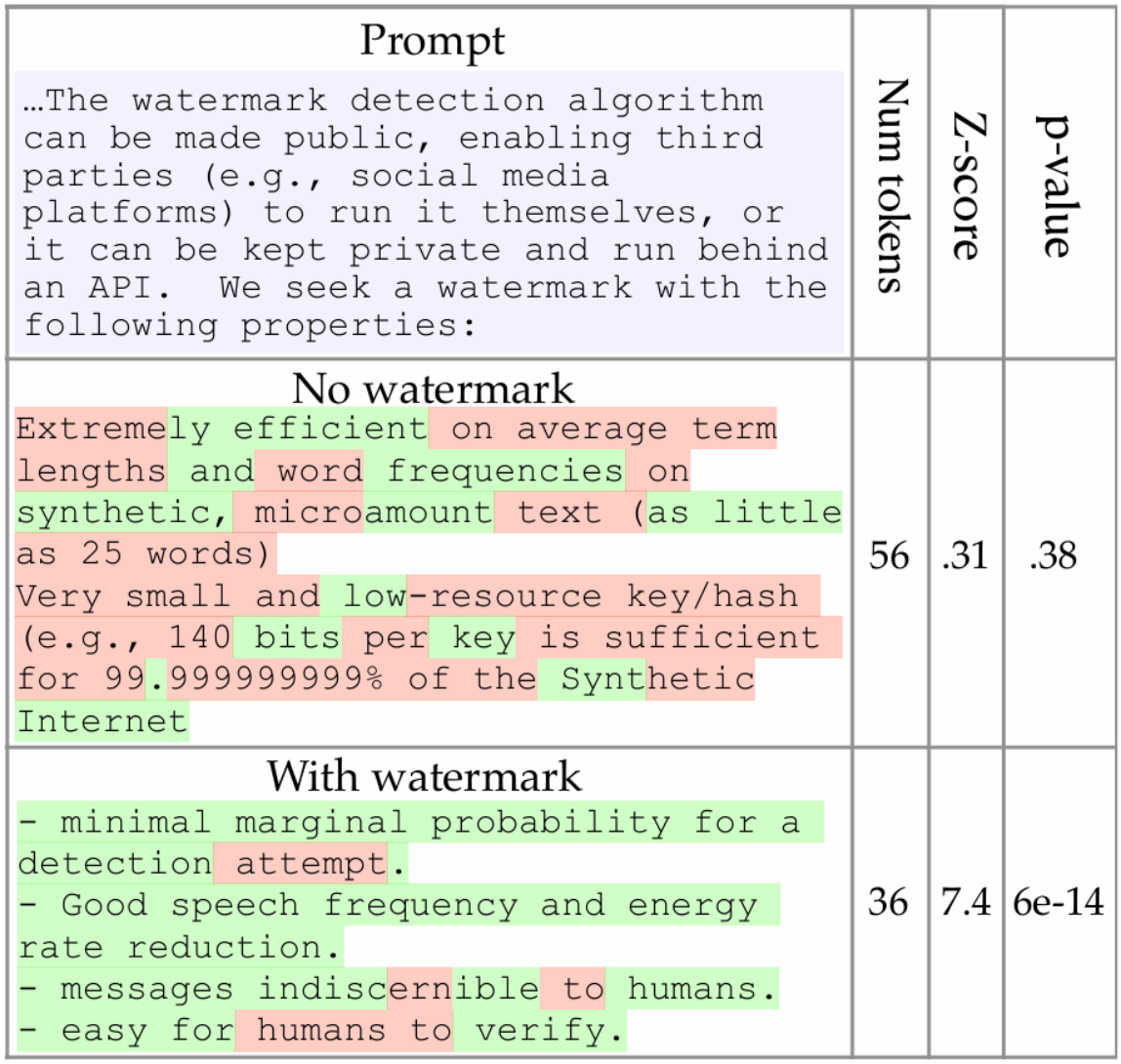


Fig. 4. Changes in text content before and after adding watermarks. (Kirchenbauer et al. 2023)

comparative, ablation, robustness and other experiments to further demonstrate the method's superiority. It will also critically examine potential challenges encountered during experimentation and propose improvements aimed at enhancing the algorithm's performance and robustness, ultimately advancing the effectiveness and applicability of digital watermarking techniques for provenance tracking and copyright protection in large language model-generated content.

2 Methodology

The framework adapted in this project builds upon the Mizero watermarking algorithm (Z. Zhang et al. 2025), with modifications tailored to the current task. The workflow of the entire watermark algorithm is shown in the Figure 5. The details of our method will be pre-

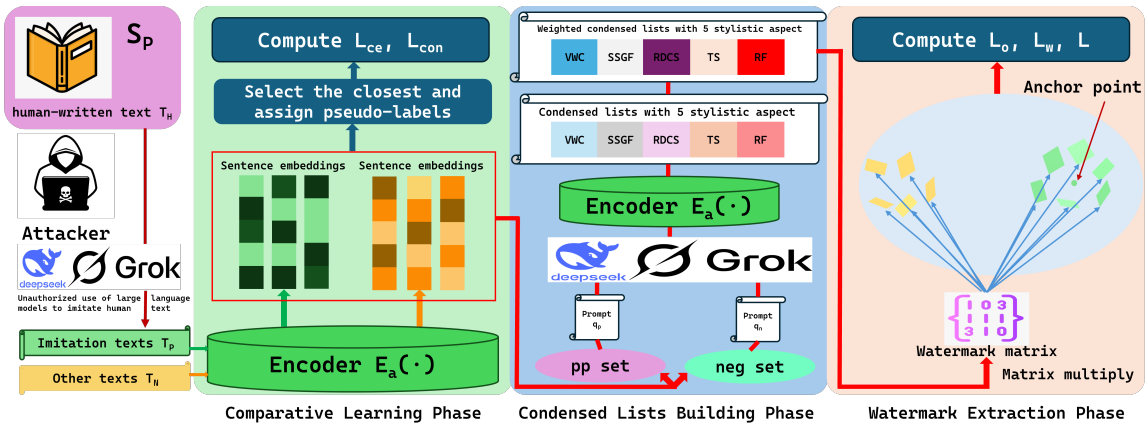


Fig. 5. The workflow of the watermark algorithm in this project.

sented in the subsequent sections. Following this, we will highlight the advantages of our method compared to other existing approaches and provide well-founded justifications for selecting this technique.

2.1 Problem Formulation

Consider a scenario where an unauthorized attacker employs a large language model (LLM) to mimic human-written text T_H of a specific style, thereby generating style-infringing content. The objective of our method is to determine, given a suspicious text, whether its style has been produced by an LLM imitating human text of the same style.

For the convenience of subsequent description, we can divide text and the style of text into the following two categories:

- **Positive class** — Formally, let the human-written text T_H possess a target style denoted as S_P. An unauthorized attacker then employs an LLM to mimic T_H, generating style-infringing text denoted as T_P.
- **Negative class** — Any text outside the protected set, denoted as T_N, including (i) human-written text whose style does not match S_P, and (ii) machine-generated text not produced to mimic S_P.

We assume that potential attackers possess two capabilities:

1. **Access to high-value datasets** (e.g., books or online news articles), enabling them to imitate the protected style.

2. **The ability to operate APIs or pipelines that conceal the internal mechanisms of such style imitation.**

With the above definition, our method objective can be described as follows. Consider a scenario where an unauthorized attacker uses a LLM to mimic T_H (style S_p), producing machine-generated text T_P. From a defensive standpoint, our objective is to design an algorithm $A(\cdot)$ that, given a suspicious text t_{test}, extracts an implicit, verifiable watermark from T_P and T_N, and compares t_{test} and T_P using the Hamming distance. If their distance is sufficiently small, we infer that t_{test} imitates the protected style.

Formally:

$$p_r = \begin{cases} 1, & \text{if } d_h\big(A(t_{\text{test}}), A(T_P)\big) < \epsilon, \\ 0, & \text{otherwise} \end{cases} \tag{3}$$

Let p_r denote the predicted probability that the test sample t_{test} is classified as style-infringing. Here, d_h represents the Hamming distance, and ϵ is empirically fixed at 1% of the watermark length.

2.2 Details of the Chosen Approach

The methodology of this project consists of three main stages. The following subsections will describe each stage in detail, starting with the Comparative Learning Phase.

2.2.1 Comparative Learning Phase

The goal of this phase is to initially distinguish T_P from T_N. In this stage, we use a contrastive learning mechanism to classify the sentence embedding vectors of T_P and T_N obtained by the encoder.

Comparative learning phase begins by extracting sentence-level embeddings for both the protected-style corpus (T_P) and the non-protected corpus (T_N). Specifically, an encoder $E_\alpha(\cdot)$, parameterized by α and can encode the input sentence into a sentence embedding

of length l_a, is employed to transform each sentence into a sequence of sentence embeddings. For every sentence $t_p \in T_P$ and $t_n \in T_N$, we obtain feature vectors $p_i \in \mathbb{R}^{1\times l_a}$ and $n_i \in \mathbb{R}^{1\times l_a}$ respectively. Assuming that both T_P and T_N consist of *num* samples each, their extracted feature vectors can be represented as the sets respectively.

$$P = \{p_1, p_2, \ldots, p_{\text{num}}\} \quad \text{and} \quad N = \{n_1, n_2, \ldots, n_{\text{num}}\} \tag{4}$$

To identify style-related similarities, we compute the cosine similarity $d(\cdot)$ between each vector $x \in P \cup N$ and all other vectors in $P \cup N$ excluding x itself, and then select the closest match, denoted as y_x^*:

$$y_x^* = \arg\max_{y \in P\cup N\setminus\{x\}} d(x, y), \quad d_x^* = d(x, y_x^*) \tag{5}$$

Then, based on the category of y_x^*, we assign a pseudo-label to x, denoted as $\hat{y}_x$. Subsequently, the cross-entropy loss between the true label y_x and the pseudo-label $\hat{y}_x$ is computed as shown in Equation (6).

$$\mathcal{L}_{\text{ce}} = \frac{1}{2 \times num} \sum_{x \in P\cup N} H(y_x, \hat{y}_x), \tag{6}$$

where $H(\cdot)$ is the entropy function.

To further enhance the distinction between positive and negative styles, we incorporate a contrastive loss:

$$\mathcal{L}_{con} = \frac{1}{2 \times num} \left(\sum_{x, x' \in P} \|x - x'\|^2 + \sum_{x \in N, x'' \in P} \max(0, m - \|x - x''\|^2) \right) \tag{7}$$

Here, m is a predefined margin that enforces a minimum separation between negative samples and positive samples, thereby encouraging discriminative representation learning.

2.2.2 Condensed Lists Building Phase

In the previous comparative learning phase, we obtained sentence-level style feature vectors using an encoder $E_\alpha(\cdot)$ and distinguished them through contrastive learning. However, contrastive learning alone is insufficient to effectively capture the nuanced stylistic differences between texts. Therefore, in this phase, we leverage a LLM to extract richer style-

representative feature vectors, aiming to better encode the stylistic attributes of sentences for improved differentiation.

Firstly, for each sentence $t_m \in T_P \cup T_N$, we concatenate it with the pre-designed prompt q to form $q \| t_m$, which is then input as a single sequence into the large language model, denoted as LLM$(\cdot)$. The purpose of this step is to allow LLM to extract the relevant style features of t_m, and the output is a natural language text.

Regarding prompt design, we partially adapt the prompts used in the Mizero algorithm (Z. Zhang et al. 2025), where the prompt requires the LLM to extract stylistic features of sentences across five dimensions: vocabulary and word choice (VWC), syntactic structure and grammatical features (SSGF), rhetorical devices and stylistic choices (RDCS), tone and sentiment (TS), and rhythm and flow (RF). However, during practical experimentation, we observed that merely extracting and combining these five stylistic aspects into a single sentence does not effectively emphasize the distinct contributions of each dimension. Therefore, we modified the original prompt such that the LLM outputs the five stylistic dimensions as five separate sentences. This design provides a foundation for applying an attention mechanism to weight each stylistic feature independently in subsequent stages.

Moreover, inspired by prompt-engineering research (Sahoo et al. 2024), to improve performance in this phase, the sentence embeddings obtained from the previous step are partitioned into positive (pp) and negative (neg) sets according to the following rules:

$$pp = \{(x, y_x^*) \mid x \in P \cup N,\, y_x^* \in P,\, d_x^* > \sigma\} \tag{8}$$

$$neg = \{x \mid x \in P \cup N,\, (y_x^* \in N) \vee (d_x^* \leq \sigma)\} \tag{9}$$

Here, σ is a predefined threshold. In practice, pp captures samples closely resembling the protected style, paired with their most relevant prior knowledge, while neg contains samples that are stylistically diverse or insufficiently similar.This partitioning allows more effective selection of optimal prior knowledge for each sample.

Finally, since sentence embeddings in the set pp appear in pairs while those in neg exist individually, we designed different prompt templates for sentences from the different sets.

For further details regarding the prompt templates, please refer to Appendix A.

The generation of the condensed list can be formally written as:

$$[s_i] = \text{LLM}(q \| t_m), \quad i = 1, 2, 3, 4, 5 \tag{10}$$

where each $\text{s}_i(\cdot)$ corresponds to extracting the i-th stylistic aspect. For every sentence, we obtain a condensed list $[s_1], [s_2], [s_3], [s_4], [s_5]$ which, as previously mentioned, represent five distinct aspects of the original sentence t_m's style.

Next, these five sentences are encoded by the encoder $E_\alpha(\cdot)$ to produce a style matrix$M_S \in \mathbb{R}^{5 \times l_a}$. It should be emphasized that this encoder is identical to the one employed during the initial feature extraction stage. Then, we apply an attention matrix $M_{Att} \in \mathbb{R}^{1 \times 5}$ to weight each row vector of the style matrix. The attention-weighted style matrix $M_{SAtt} \in \mathbb{R}^{5 \times l_a}$ is computed as follows.

$$M_{SAtt} = softmax(M_{Att}) \times M_S \tag{11}$$

In Equation (11), $softmax$ is applied row-wise to M_{Att}, producing attention weights that sum to one for each row. The symbol $\times$ denotes standard matrix multiplication, allowing these attention weights to be applied to the style feature matrix M_S.

2.2.3 Watermark Extraction Phase

In this Phase, we introduce a learnable watermark matrix $M_w \in \mathbb{R}^{l_a \times len_w}$. After obtaining M_{SAtt} in the previous stage, we multiply it by M_w and subsequently apply a sigmoid function to the result. This operation maps the condensed list of textual style features into a new feature space. The process is formally defined as follows:

$$w = sigmoid\big(M_{SAtt} \times M_w\big), \tag{12}$$

In this context, the vector $w \in \mathbb{R}^{5 \times len_w}$ represents the style feature vector mapped into the new style feature space for a given sentence. After obtaining the style feature vectors w_i for all sentences t_i, where $t_i \in T_P \cup T_N$, we assume that all sentences from T_P, once mapped into the new style feature space, will converge to a single anchor point. We de-

note this anchor point as a, which is computed as follows:

$$a = \frac{1}{l} \sum_{i=1}^{l} w_i, \tag{13}$$

where l denotes the number of sentence embedding pairs in the pp set.

To better quantify the convergence of w_i, we introduce a new loss function L_o, defined as follows:

$$\mathcal{L}_o = \frac{1}{l} \sum_{i=1}^{l} \|w_i - a\|^2, \tag{14}$$

After obtaining the style anchor point a for T_P, our proposed method concludes its feature extraction process. In the following sections, we describe the training procedure for optimizing the model parameters, as well as the approach for utilizing the style anchor point a in watermark verification of suspicious texts.

2.3 Training Process

For each sentence $t_m \in T_P \cup T_N$, we collect the set of watermark vectors corresponding to protected style texts as $W_P = \{w_m \mid t_m \in P\}$. To further measure the effectiveness of the encoder $E_\alpha(\cdot)$ and the watermark matrix M_w, we first aggregate the set W_P into a single representative vector. One simple approach is to compute the mean vector over all elements in W_P, denoted as $\bar{w}_P$. We then apply the Binary Cross-Entropy (BCE) loss between this aggregated vector $\bar{w}_P$ and the style anchor point a:

$$\mathcal{L}_w = \text{BCELoss}(\bar{w}_P, a) \tag{15}$$

Here, a is a fixed style anchor vector, and $\bar{w}_P$ summarizes the watermark information from all sentences in the protected set. This formulation ensures that the BCE loss is computed between two vectors of the same shape.

Consequently, the overall loss function is defined as:

$$\mathcal{L} = \mathcal{L}_{ce} + \mathcal{L}_{con} + \mathcal{L}_w + \mathcal{L}_o \tag{16}$$

Algorithm 1: Training Procedure

Input: Protected style S_P, imitation texts T_P, unprotected texts T_N, encoder $E_\alpha(\cdot)$, cosine similarity $d(\cdot)$, watermark matrix M_w, LLM $LLM(\cdot)$, prompts q_p, q_n, R and ep epochs

Output: Updated parameters α, γ

for $epoch \leftarrow 1$ **to** ep **do**

  foreach $t_m \in T_P \cup T_N$ **do**

    $x \leftarrow E_\alpha(t_m)$;

    $y_x^* \leftarrow \underset{y \in P \cup N \setminus \{x\}}{\arg\max}\ d(x, y)$;

  Compute $\mathcal{L}_{ce}$ (Eq. 6) and $\mathcal{L}_{con}$ (Eq. 7);

  Construct pp and neg using Eq. (8) and Eq. (9);

  foreach $t_m \in T_P \cup T_N$ **do**

    $$[s_{m1}], [s_{m2}], [s_{m3}], [s_{m4}], [s_{m5}] \leftarrow \begin{cases} LLM(q_p \mid t_m), & \text{if } t_m \in pp \\ LLM(q_n \mid t_m), & \text{if } t_m \in neg \end{cases};$$

    $M_{Sm} = E_\alpha([s_{m1}], [s_{m2}], [s_{m3}], [s_{m4}], [s_{m5}])$;

    Compute M_{SAtt_m} using Eq. (11);

    Compute w_m using Eq. (12);

    Calculate a (Eq. 13) , $\mathcal{L}_o$ (Eq. 14) and $\mathcal{L}_w$ (Eq. 15);

    Calculate $\mathcal{L}$ (Eq. 16);

  Update E_α, M_{SAtt_m} with the overall loss L;

2.4 Testing Process

The testing procedure largely mirrors the training process. Given a suspicious test text containing a sentence t_{test}, we first classify t_{test} into either the pp set or the neg set using the operations defined in the comparative learning phase. Subsequently, following the condensed list building phase, we generate the condensed list by leveraging the LLM with the optimal combined input $q \| t_{\text{test}}$:

$$[s_{test1}], [s_{test2}], [s_{test3}], [s_{test4}], [s_{test5}] \leftarrow \begin{cases} LLM(q_p \mid t_{test}), & \text{if } t_{test} \in pp \\ LLM(q_n \mid t_{test}), & \text{if } t_{test} \in neg \end{cases} \tag{17}$$

This enables us to obtain the style feature vector corresponding to t_{test}, denoted as w_{test}, through the same mapping procedure.

After obtaining the style feature vector w_{test}, we perform watermark verification to determine whether t_{test} imitates the protected style S_P. This verification follows a procedure analogous to Equation (3), and is formally expressed as:

$$p_r = \begin{cases} 1, & \text{if } d_h\big(round(w_{\text{test}}), round(a)\big) < \epsilon, \\ 0, & otherwise, \end{cases} \tag{18}$$

Here, the function round$(\cdot)$ converts the continuous feature vectors into binary form to enable discrete comparison via the Hamming distance $d_h(\cdot)$. Note that the reference vector a used here is consistent with the style anchor point vector a obtained during the training process.

2.5 Reasons for Method Selection

The above method was deliberately chosen to address the limitations observed in existing watermarking approaches, particularly those that rely on the “green list” concept. In green list–based methods, the watermark signal is embedded by increasing the sampling probability of tokens from a predefined subset. While this approach can achieve detectable watermark signals, it inevitably biases the token distribution, often altering the content of the generated text. This alteration compromises the naturalness, coherence, and stylistic fidelity of the text.

In addition, methods that rely on the “green list” concept typically require extensive computation to obtain reliable statistical results, as they depend on analyzing large amounts of generated text to estimate the proportion of green-listed tokens. In contrast, our approach achieves high detection accuracy with only a small number of instances, enabling a *few-shot* capability. This significantly reduces computational resource consumption, making the method more efficient and practical for real-world deployment.

Moreover, our attention matrix is designed to assign weights to five distinct aspects of tex-

tual style. These learned weights allow us to interpret which style dimensions contribute most to the protected style representation, thereby providing strong model interpretability. By contrast, the “green list” method merely computes the proportion of green tokens, offering limited insight into the underlying stylistic features and their relative importance.

Finally, Table 1 summarizes the comparison between our method and representative alternatives based on the “green list” concept.

Table 1. Comparison of the chosen Method and Green List–Based Watermarking Methods.

Criteria	**Chosen Method (Non-Green List)**	**Green List–Based Methods**
Watermark Embedding Mechanism	Implicit style anchor points, no token bias	Increase sampling probability of green tokens
Impact on Text Quality	Minimal alteration, preserves naturalness and style	Alters token distribution, reduces naturalness
Computational Efficiency	Few-shot, requires only a small number of samples	Requires large-scale statistical computations
Interpretability	Attention weights on five style aspects, providing insight	Relies on green token ratio, limited interpretability

3 Evaluation and Reflection

3.1 Watermark Detection Experiment

Since the main goal of our method is to protect the style of a given text through watermark detection, preventing it from being abused by unauthorized attackers using large language models, the success rate of watermark detection is crucial to our method. In this section, experiments will be carried out to compare the success rate of watermark detection of our method with other methods.

3.1.1 Evaluation Metrics

We focus on three key metrics: True Positive Rate (TPR), False Positive Rate (FPR), and F1 score (F1). The computation of these metrics is given as follows:

$$TPR = \frac{TP}{TP + FP} \tag{19}$$

$$FPR = \frac{FP}{FP + TN} \tag{20}$$

$$F1 = \frac{2 \times TP}{2 \times TP + FP + FN} \tag{21}$$

where TP denotes the number of cases in which the model successfully detects a suspicious text t_{test} that imitates style S_P, FP denotes the number of cases where the model incorrectly predicts that a text t_{test} imitates style S_P when it does not, FN denotes the number of unwatermarked texts that are not detected, and TN denotes the number of correctly identified unwatermarked texts.

These three metrics respectively correspond to the performance of the watermark detection model in protecting textual style, avoiding false alarms, and achieving an overall balance between precision and recall. By jointly considering TPR, FPR, and F1, we can comprehensively evaluate the capability of the watermark detection model in effectively safeguarding the protected text style.

3.1.2 Datasets

In the data preparation process, we employed two publicly available modern text datasets with distinct stylistic characteristics: ROC (Zhu et al. 2022) and New York Times Articles Metadata 2020_2022 (Schatz 2022) (hereinafter referred to as NYT). In the ROC dataset, the texts are presented in the form of short stories, whereas in the NYT dataset, the texts are structured as news reports. Consequently, the ROC dataset exhibits a more colloquial linguistic style, while the NYT dataset displays a more formal, written style. Moreover, individual sentences in the ROC dataset tend to be shorter in length compared to those in the NYT dataset, where sentences are generally longer and syntactically more complex.

To construct the *TP* and *TN* sets, we utilized a large language model (LLM) to rewrite the original texts from these datasets according to specific style imitation.

The detailed composition of the datasets we constructed, including the number of samples and the average number of tokens, is summarized in Table 2.

Table 2. Statistics of the employed dataset.

	S_P	Other Styles	**DeepSeek**		**Grok**	
			Size	AVG_token	Size	AVG_token
Train	NYT	ROC	300	18.4	300	17.6
	ROC	NYT	300	14.6	300	15.1
Test	NYT	ROC	140	17.9	140	18.1
	ROC	NYT	140	15.2	140	14.4

3.1.3 Experimental Setting

In this experiment, we select state-of-the-art watermarking algorithms including KGW (Kirchenbauer et al. 2023), EWD (Lu et al. 2024b), SynID (Dathathri et al. 2024) and Unigram (Zhao et al. 2023) to compare with the proposed method.

Unless otherwise specified, all experiments were conducted under the following settings.

All experiments were conducted under the following hardware and software settings:

Table 3. Experimental Environment.

Component	**Specification**
CPU	Intel Core i5-13500H @ 2.6 GHz
GPU	Intel Iris Xe Graphics
RAM	16 GB 2600 MHz
OS	Windows 11 24H2
Python	3.8.20
PyTorch	2.4.1

In this experiment, all non-green list watermarking methods were implemented using the MarkLLM (Pan et al. 2024) toolkit, with PyTorch version 3.10.18.

Regarding the sentence encoder, we utilize the `sup-simcse-bert-base-uncased` variant of SimCSE (T. Gao, Yao, and Chen 2021) throughout this project. We adopt the AdamW optimizer(Loshchilov and Hutter 2017), applying different learning rates for the sentence encoder, the watermark matrix, and the attention matrix. The specific parameter settings are summarized in Table 4.

Table 4. Learning rate setting.

Parameters	Learning rate
sentence encoder	5e-5
watermark matrix	1e-5
attention matrix	1e-4

We employ DeepSeek as the default method to imitate the protected style S_P for text generation. We consistently employ the same large language model for generating the condensed lists as the one used to produce the imitation texts. Imitation texts are generated by randomly sampling 20 instances. Experimental results are reported as the average values obtained over five independent runs. Detailed hyperparameter settings for SOTA watermarking methods can be found in the Appendix B.

3.1.4 Experimental Results

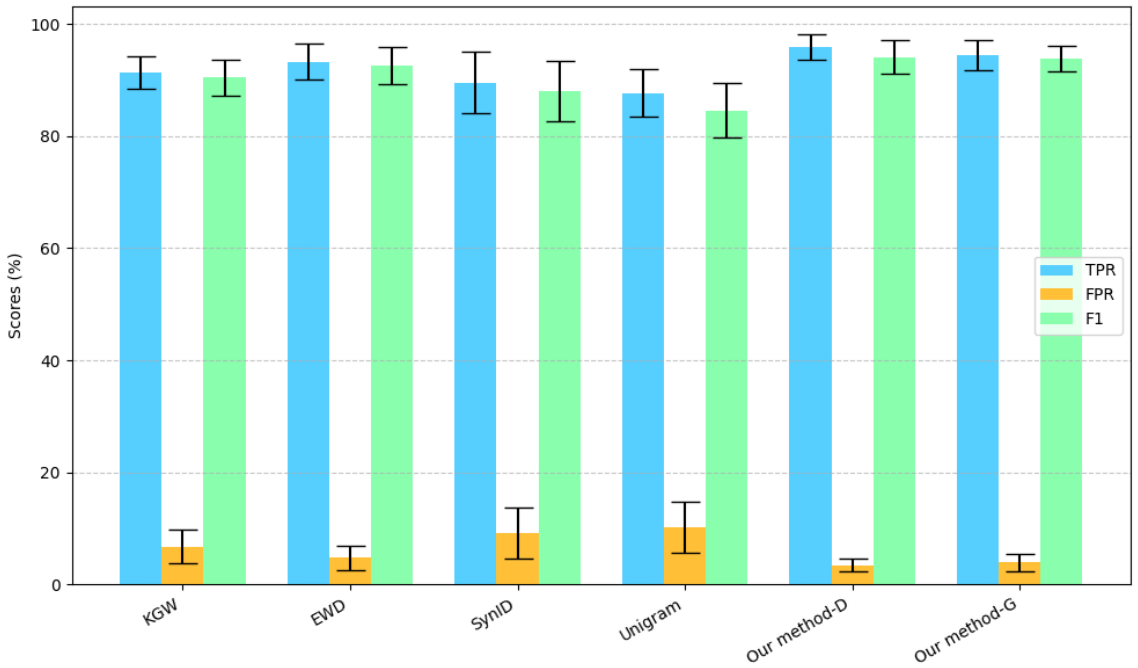
The following table presents the TPR, FPR, and F1 metrics for the comparison of our method with state-of-the-art watermarking algorithms. Specifically, in the first row, "DeepSeek" and "Grok" refer to the specific LLMs used to generate the condensed lists. "Our method-D" refers to the case where "DeepSeek" is used to generate the imitation texts, while "Our method-G" refers to the case where "Grok" is used to generate the imitation texts. Results are reported as the mean $\pm$ standard deviation.

Table 5. Comparison of our method with SOTA watermarking algorithms.

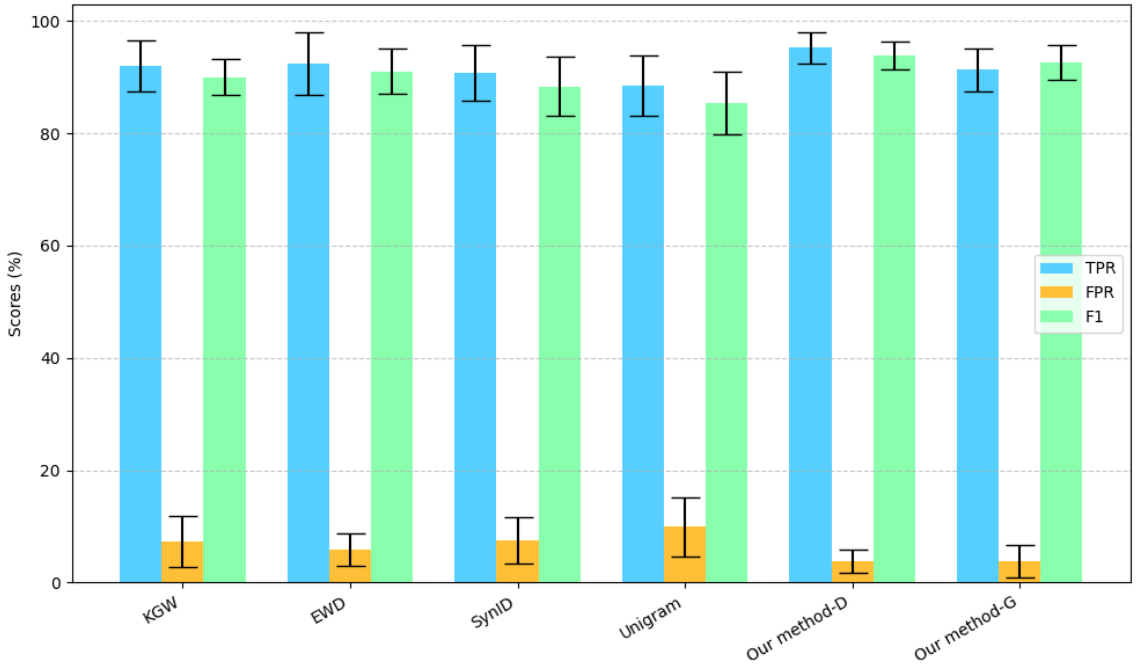
Methods	**DeepSeek**			**Grok**		
	TPR	FPR	F1	TPR	FPR	F1
KGW	$91.31_{\pm 2.86}$	$6.73_{\pm 3.02}$	$90.45_{\pm 3.17}$	$92.06_{\pm 4.51}$	$7.31_{\pm 4.50}$	$90.01_{\pm 3.15}$
EWD	$93.25_{\pm 3.22}$	$4.77_{\pm 2.16}$	$92.51_{\pm 3.35}$	$92.49_{\pm 5.56}$	$5.83_{\pm 2.92}$	$90.98_{\pm 4.03}$
SynID	$89.53_{\pm 5.43}$	$9.14_{\pm 4.57}$	$87.92_{\pm 5.37}$	$90.74_{\pm 4.88}$	$7.44_{\pm 4.12}$	$88.37_{\pm 5.22}$
Unigram	$87.67_{\pm 4.17}$	$10.22_{\pm 4.63}$	$84.53_{\pm 4.90}$	$88.53_{\pm 5.32}$	$9.91_{\pm 5.18}$	$85.42_{\pm 5.64}$
Our method-D	$95.89_{\pm 2.31}$	$3.47_{\pm 1.21}$	$94.11_{\pm 3.02}$	$95.22_{\pm 2.75}$	$3.76_{\pm 2.05}$	$93.87_{\pm 2.51}$
Our method-G	$94.42_{\pm 2.63}$	$3.94_{\pm 1.55}$	$93.78_{\pm 2.28}$	$91.31_{\pm 3.86}$	$3.83_{\pm 2.82}$	$92.64_{\pm 3.12}$

Figure 6 provides a more intuitive visualization of the experimental results presented in Table 5.

From the comprehensive results presented in Table 5 and illustrated visually in Figure 6, our proposed method consistently and significantly outperforms several well-established



(a) Performance of generating condensed lists using DeepSeek.



(b) Performance of generating condensed lists using Grok.

Fig. 6. Comparison of Our Method with State-of-the-Art Watermarking Algorithms: Bar Charts with Error Bars.

baseline watermarking techniques, including KGW, EWD, SynID, and Unigram, across all evaluated metrics: True Positive Rate (TPR), False Positive Rate (FPR), and F1 score. This superior performance highlights the robustness and effectiveness of our approach in accurately detecting textual watermarks embedded within imitation texts generated by large language models (LLMs).

More specifically, our method achieves the highest TPR and F1 scores on both the DeepSeek and Grok datasets, which represent different LLM-generated imitation text sources. This strong performance demonstrates the ability of our approach to reliably identify imitation texts, thereby providing effective protection against unauthorized textual content replication and style imitation. The reduction in false positive rates (FPR) further emphasizes our method's ability to minimize incorrect detections, which is crucial for maintaining trustworthiness in real-world applications where false alarms could cause operational inefficiencies or user dissatisfaction. We believe this is mainly because our method maps style-related features into an implicit zero-watermark, thereby removing the necessity for embedding during the generation process. This design choice preserves the highest level of style integrity while ensuring robust compatibility with detection mechanisms across diverse generative models.

Additionally, the relatively low variance in performance metrics across repeated experimental runs indicates that our method yields stable and reproducible results. Such stability is a desirable property, ensuring that the watermark detection remains consistent regardless of minor fluctuations in input data or environmental conditions. This robustness supports the practical deployment of our method in diverse and dynamic settings.

An interesting observation from our experiments is that the variant of our method using imitation texts generated by DeepSeek ("Our method-D") slightly surpasses the variant using Grok ("Our method-G") in most metrics. Nevertheless, both variants maintain a clear and substantial advantage over the existing watermarking algorithms, underscoring the generalizability of our technique across different large language models and imitation text sources. This cross-model adaptability suggests that our method does not overly depend on the characteristics of any single LLM, enhancing its applicability and scalability in varied scenarios.

In summary, these comprehensive experimental results strongly validate the efficacy and superiority of our proposed watermarking approach for textual style protection. Our method

not only achieves high accuracy in watermark detection but also demonstrates remarkable robustness and stability, positioning it as a promising and practical solution for defending textual content style against unauthorized imitation and ensuring content authenticity in the era of advanced language generation technologies.

3.2 Few-shot Evaluation Experiments

As mentioned earlier, our method has the few-shot property. Specifically, it can achieve good watermark detection performance with only a few sentence examples. In this section, we will demonstrate this through experimental results.

3.2.1 Experimental Setting

To demonstrate the few-shot capability of our method, this section of experiments controls all other parameters constant while varying only the number of samples *num* drawn from T_H (the protected human-written text). This setup allows us to explore whether our method can maintain strong performance even when *num* is sufficiently small. The evaluation metrics used to assess the method's performance remain consistent with those in the previous section, namely True Positive Rate (TPR), False Positive Rate (FPR), and F1 score. The dataset used in this section is the same as that in the previous section. Other experimental parameter settings are the same as in the previous section.

3.2.2 Experimental Results

Table 6 and Figure 7 illustrate the variation in the performance metrics (TPR, F1, and FPR) of our proposed method under different values of *num*.

As shown in Table 6 and Figure 7, a clear trend emerges with respect to the number of samples, denoted as *num*, drawn from T_H. First, for all evaluation metrics—namely TPR, FPR, and F1 score—our proposed approach demonstrates a consistent improvement as *num* increases. This indicates that the method is capable of leveraging even marginal increments in the available reference samples to enhance both detection accuracy and overall robustness.

Table 6. Few-shot evaluation experiments results.

num	Methods	DeepSeek			Grok		
		TPR	FPR	F1	TPR	FPR	F1
5	Our method-D	$89.25_{\pm 3.69}$	$6.73_{\pm 2.51}$	$88.75_{\pm 3.23}$	$88.96_{\pm 3.91}$	$6.71_{\pm 3.03}$	$88.94_{\pm 3.35}$
	Our method-G	$88.67_{\pm 3.99}$	$7.83_{\pm 4.04}$	$86.23_{\pm 3.53}$	$88.57_{\pm 4.51}$	$6.97_{\pm 2.08}$	$88.03_{\pm 3.22}$
10	Our method-D	$92.33_{\pm 3.85}$	$5.37_{\pm 2.82}$	$91.53_{\pm 3.67}$	$90.95_{\pm 4.51}$	$6.23_{\pm 3.57}$	$89.62_{\pm 4.05}$
	Our method-G	$91.51_{\pm 3.17}$	$5.87_{\pm 2.42}$	$90.98_{\pm 2.95}$	$90.26_{\pm 3.70}$	$4.93_{\pm 2.06}$	$90.55_{\pm 3.46}$
15	Our method-D	$95.01_{\pm 3.14}$	$3.77_{\pm 1.95}$	$93.62_{\pm 3.44}$	$94.16_{\pm 4.75}$	$4.31_{\pm 2.09}$	$93.01_{\pm 3.15}$
	Our method-G	$93.28_{\pm 3.13}$	$4.03_{\pm 2.41}$	$92.45_{\pm 2.79}$	$91.13_{\pm 3.44}$	$4.21_{\pm 2.30}$	$92.15_{\pm 3.01}$
20	Our method-D	$95.89_{\pm 2.31}$	$3.47_{\pm 1.21}$	$94.11_{\pm 3.02}$	$95.22_{\pm 2.75}$	$3.76_{\pm 2.05}$	$93.87_{\pm 2.51}$
	Our method-G	$94.42_{\pm 2.63}$	$3.94_{\pm 1.55}$	$93.78_{\pm 2.28}$	$91.31_{\pm 3.86}$	$3.83_{\pm 2.82}$	$92.64_{\pm 3.12}$
	KGW	$91.31_{\pm 2.86}$	$6.73_{\pm 3.02}$	$90.45_{\pm 3.17}$	$92.06_{\pm 4.51}$	$7.31_{\pm 4.50}$	$90.01_{\pm 3.15}$
	EWD	$93.25_{\pm 3.22}$	$4.77_{\pm 2.16}$	$92.51_{\pm 3.35}$	$92.49_{\pm 5.56}$	$5.83_{\pm 2.92}$	$90.98_{\pm 4.03}$
	SynID	$89.53_{\pm 5.43}$	$9.14_{\pm 4.57}$	$87.92_{\pm 5.37}$	$90.74_{\pm 4.88}$	$7.44_{\pm 4.12}$	$88.37_{\pm 5.22}$
	Unigram	$87.67_{\pm 4.17}$	$10.22_{\pm 4.63}$	$84.53_{\pm 4.90}$	$88.53_{\pm 5.32}$	$9.91_{\pm 5.18}$	$85.42_{\pm 5.64}$

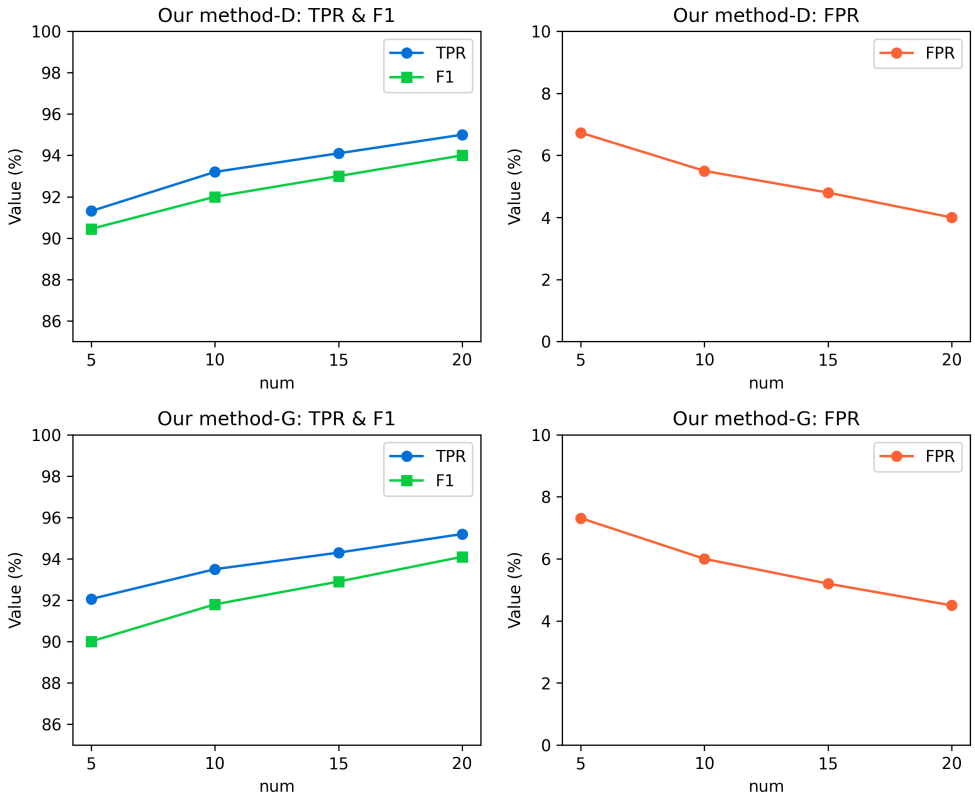


Fig. 7. Changes in text content before and after adding watermarks. (Kirchenbauer et al. 2023)

More importantly, the experimental results highlight the method's strong *few-shot* capability. Specifically, when only five sentences are sampled (*num* = 5), our approach attains a performance level that is broadly comparable to that of the Unigram baseline. This is a particularly noteworthy observation, as it suggests that our framework can maintain competitive performance even under conditions of extremely limited data availability, thereby demonstrating its potential applicability to low-resource or real-time scenarios.

Furthermore, when the sampling size is increased slightly to ten sentences (*num* = 10), the performance of our method approaches that of more sophisticated baselines such as KGW and SynID. Although it remains marginally lower than that of the EWD algorithm, the performance gap is relatively small. Considering the minimal input required, this result underscores the efficiency of our approach in capturing distinctive stylistic signals from very few exemplars.

In summary, the evidence from both tabular and graphical analyses substantiates that the proposed method is not only scalable with increasing *num* but also retains strong discriminative capability under highly constrained sampling conditions. This combination of scalability and efficiency makes it well-suited for deployment in scenarios where access to large-scale human-written reference corpora is impractical or infeasible.

3.3 Robustness Evaluation Experiments

Although the proposed approach has already demonstrated outstanding watermark detection capabilities in both the "Watermark Detection Experiment" and "Few-shot Evaluation Experiments" sections, it is important to examine its robustness under more challenging conditions. As noted by Dugan et al. in the RAID Benchmark study, many existing machine-generated text detection algorithms achieve impressive performance on standard datasets; however, their effectiveness often degrades substantially when the input text is subjected to adversarial attacks (Dugan et al. 2024). This phenomenon highlights a critical gap in current evaluation practices, as real-world applications may encounter inputs that have been intentionally manipulated to evade detection.

3.3.1 Experimental Setting

Motivated by this observation, we extend our evaluation to investigate the resilience of the proposed method against various adversarial attacks. The objective is to determine whether our approach can retain satisfactory performance levels even when confronted with deliberately altered text. Specifically, we consider the following adversarial strategies:

1. **Number**: Randomly shuffling the digits within numerical values.
2. **Upper-Lower**: Swapping the case of letters within words.
3. **Add Paragraph**: Inserting the “\n\n” line break between sentences.
4. **Article Deletion**: Removing common English articles such as “the,” “a,” and “an.”
5. **Synonym**: Replacing tokens with highly similar candidate tokens identified using BERT (Devlin et al. 2019).

Regarding the application range of each type of adversarial attacks, we apply a specific adversarial attack to 20% of the total text length. By systematically applying these adversarial attacks, we aim to rigorously assess the extent to which the proposed method maintains its detection accuracy in the presence of adversarial manipulation. This evaluation not only offers a deeper understanding of our model’s robustness but also provides valuable insights into the broader challenge of securing watermark-based detection systems against intentional evasion.

3.3.2 Experimental Results

Table 7 presents the results of the robustness evaluation experiments under five types of adversarial attacks: Number, Upper-Lower, Add Paragraph, Article Deletion, and Synonym. For each attack, we report the True Positive Rate (TPR), False Positive Rate (FPR), and F1-score. The values following the arrows represent the performance differences relative to ”Our method-D.”

Based on the experimental results presented in the table, it is evident that the ”Synonym” type of adversarial attack causes the most significant degradation in the performance of

Table 7. Robustness evaluation experiments results.

Adversarial Attacks	DeepSeek			Grok		
	TPR	FPR	F1	TPR	FPR	F1
Number	$94.95_{\downarrow 0.94}$	$3.84_{\downarrow 0.37}$	$93.80_{\downarrow 0.31}$	$94.76_{\downarrow 0.46}$	$3.92_{\downarrow 0.16}$	$93.52_{\downarrow 0.35}$
Upper-Lower	$94.51_{\downarrow 1.38}$	$3.89_{\downarrow 0.42}$	$93.57_{\downarrow 0.54}$	$94.13_{\downarrow 1.09}$	$4.01_{\downarrow 0.25}$	$93.35_{\downarrow 0.52}$
Add Paragraph	$93.44_{\downarrow 2.45}$	$4.02_{\downarrow 0.55}$	$92.79_{\downarrow 1.32}$	$94.04_{\downarrow 1.18}$	$3.97_{\downarrow 0.21}$	$93.36_{\downarrow 0.51}$
Article Deletion	$91.08_{\downarrow 4.81}$	$5.68_{\downarrow 2.21}$	$90.43_{\downarrow 3.68}$	$91.94_{\downarrow 3.28}$	$4.51_{\downarrow 0.75}$	$90.76_{\downarrow 3.11}$
Synonym	$89.77_{\downarrow 6.12}$	$7.41_{\downarrow 3.94}$	$88.64_{\downarrow 5.47}$	$90.32_{\downarrow 4.90}$	$6.85_{\downarrow 3.09}$	$89.51_{\downarrow 4.36}$
Our method-D	95.89	3.47	94.11	95.22	3.76	93.87

our method. The next most impactful attack is the ”Article Deletion” type. In contrast, the ”Number” and ”Add Paragraph” attacks result in relatively minor decreases in performance. This can be attributed to the fact that, compared to other adversarial attack types, the ”Synonym” and ”Article Deletion” attacks alter the actual content of the text more substantially, thereby exerting a more substantial influence on the textual style. Although ”Add Paragraph” and ”Upper-Lower” attacks modify the text by inserting line breaks or changing letter cases, these modifications do not alter the underlying meaning of the sentences, resulting in a comparatively minor effect on text style. Finally, the ”Number” attack has a minimal impact because most texts either do not contain numbers or include them only sparingly, so modifications to numbers affect only a small portion of the text, thus having a negligible influence on the performance of our method. Overall, these observations underscore that adversarial attacks targeting semantic content tend to have a much greater detrimental effect than those focusing solely on superficial formatting changes. This suggests that the robustness of our method is closely tied to the preservation of semantic consistency within the input text. In practical scenarios, systems should be designed to prioritize resistance against meaning-altering perturbations, as they pose a more substantial threat to performance.

3.4 Ablation Experiments

In the “Methodology” section, our proposed approach was divided into multiple sequential stages, with several distinct loss functions incorporated throughout the process. To gain a deeper understanding of the contribution of each component to the overall performance, we conducted a series of ablation experiments. These experiments were specifically designed to isolate and evaluate the impact of individual modules and loss functions, thereby

enabling a comprehensive assessment of their respective roles in achieving the final results.

3.4.1 Experimental Setting

In this experiment, we mainly designed the following experimental types:

1. $-\mathcal{L}_{\text{con}}$: Removing the contrastive loss function in the comparative learning phase.
2. $-\mathcal{L}_{\text{o}}$: Removing the $-\mathcal{L}_{\text{o}}$ loss function in the watermark extraction phase.
3. Froze $E_\alpha(\cdot)$: The encoder parameters do not change during training.
4. - Att: Removing the attention matrix and directly applying the encoded vector of the condensed list to the watermark matrix.
5. - C: Skipping condensed lists building phase. Directly apply the feature vector output by the encoder to the watermark matrix.
6. - Contrastive learning: Do not divide the feature vectors into pp and neg sets through contrastive learning.

3.4.2 Experimental Results

Table 8 shows the results of the experiment. The values following the arrows represent the performance gap relative to "Our method-D."

Table 8. Ablation experiments results.

	DeepSeek			**Grok**		
	TPR	FPR	F1	TPR	FPR	F1
$-\mathcal{L}_{\text{con}}$	$89.34_{\downarrow 6.55}$	$4.58_{\downarrow 1.11}$	$90.29_{\downarrow 3.82}$	$91.60_{\downarrow 3.62}$	$5.65_{\downarrow 1.89}$	$90.75_{\downarrow 3.12}$
$-\mathcal{L}_{\text{o}}$	$87.71_{\downarrow 8.18}$	$7.07_{\downarrow 3.60}$	$88.92_{\downarrow 5.19}$	$88.79_{\downarrow 6.43}$	$6.96_{\downarrow 2.93}$	$89.80_{\downarrow 4.07}$
Froze $E_\alpha(\cdot)$	$84.30_{\downarrow 11.59}$	$11.18_{\downarrow 7.71}$	$82.12_{\downarrow 11.99}$	$83.92_{\downarrow 11.30}$	$12.42_{\downarrow 8.66}$	$81.35_{\downarrow 12.52}$
- Att	$91.51_{\downarrow 4.38}$	$5.48_{\downarrow 2.01}$	$90.19_{\downarrow 3.92}$	$92.12_{\downarrow 3.10}$	$4.22_{\downarrow 0.46}$	$91.25_{\downarrow 2.62}$
- C	$85.80_{\downarrow 10.09}$	$9.58_{\downarrow 6.11}$	$86.77_{\downarrow 7.34}$	$86.64_{\downarrow 8.58}$	$10.90_{\downarrow 7.14}$	$85.70_{\downarrow 8.17}$
- Contrastive learning	$83.43_{\downarrow 12.46}$	$12.08_{\downarrow 8.61}$	$80.48_{\downarrow 13.63}$	$84.75_{\downarrow 10.47}$	$12.60_{\downarrow 8.84}$	$82.06_{\downarrow 11.81}$
Our method-D	95.89	3.47	94.11	95.22	3.76	93.87

The results in Table 8 show that removing the contrastive learning step and building condensed lists significantly impairs the method's performance. This demonstrates the effectiveness of using contrastive learning and building condensed lists for extracting text style features. Furthermore, freezing the parameters of the encoder $E_\alpha(\cdot)$ during training also significantly impairs the method's performance, demonstrating that the encoder $E_\alpha(\cdot)$ can continuously update its parameters during training to better extract text style features. Finally, while removing the contrastive loss function, the $-\mathcal{L}_o$ loss function, and the attention matrix have a relatively small impact on the performance of our method, their impact is still significant. In general, all these results demonstrate that each module plays a crucial role in ensuring the robustness and accuracy of the proposed method, with contrastive learning, the building of condensed lists, and updating encoder parameters being particularly indispensable.

3.5 Interpretability Evaluation Experiments

As outlined in the *Methodology* section, we stated that "the attention matrix could provide strong model interpretability." However, no quantitative evidence or detailed explanation was previously provided to substantiate this claim. To address this gap, the present section conducts a series of interpretability evaluation experiments aimed at illustrating how the proposed method leverages the attention matrix to achieve interpretability. Through these experiments, we aim to demonstrate the specific mechanisms by which the attention matrix captures stylistic patterns and facilitates a transparent understanding of the model's decision-making process.

3.5.1 Experimental Setting

Since each dataset has its own unique style, we expect that the style weights of each dataset will vary. Therefore, we introduce more datasets for comparison in this section to better highlight the differences in style weights between different datasets. Specifically, in this section we introduce two additional datasets. These four datasets have different styles. The relevant information is as follows:

1. Shakespeare plays (SP) (kingburrito666 2022): This dataset contains the complete

text of William Shakespeare's plays, enabling analysis of Early Modern English language style and literary structures.

2. arXiv Dataset (submitters 2024): This dataset comprises over 1.7 million machine-readable scholarly articles spanning diverse STEM disciplines—such as physics, mathematics, computer science, and quantitative biology—providing metadata including titles, abstracts, author lists, categories, and version histories.

3. ROCStories dataset (Zhu et al. 2022): The ROCStories dataset, introduced by the Allen Institute for AI, consists of a large collection of five-sentence short stories, where the first four sentences are given and the final sentence has both a correct and an incorrect ending. It is widely used to evaluate models on narrative understanding and commonsense reasoning tasks.

4. New York Times Articles Metadata 2020_2022 (NYT) (Schatz 2022): This dataset contains metadata for 153,059 articles published by *The New York Times* between January 2020 and October 2022, offering a comprehensive resource for analyzing recent trends and topics in a globally recognized newspaper.

These four datasets come from a wide range of sources and styles, including opera, academic papers, everyday stories, and news. For example sentences from each dataset, please refer to the appendix C. For each dataset, we randomly sampled 20 sentences as the protected human-written text (T_H) and then used LLM to generate 300 imitation sentences for training and testing. The weights of the resulting attention matrix were extracted for comparison. To ensure fair comparison, we kept all other hyperparameters consistent.

3.5.2 Experimental Results

Table 9 and Figure 9 summarize the attention weight allocations for each dataset across five stylistic dimensions, highlighting the relative emphasis of vocabulary, syntax, rhetorical devices, tone, and rhythm in shaping their distinctive textual styles.

Figure 8 capture the main stylistic features of the four datasets across five key dimensions: vocabulary and word choice (VWC), syntactic structure and grammatical features (SSGF), rhetorical devices and stylistic choices (RDCS), tone and sentiment (TS), and rhythm and flow (RF). By observing the data points in Figure 8, we can find that Shakespeare's works

Table 9. Attention weight distribution across five stylistic dimensions for different datasets.

Dataset	VWC	SSGF	RDCS	TS	RF
Shakespeare	0.28	0.22	0.25	0.15	0.10
ArXiv	0.25	0.40	0.06	0.04	0.25
ROCStories	0.22	0.18	0.15	0.32	0.13
NYT	0.27	0.28	0.12	0.18	0.15

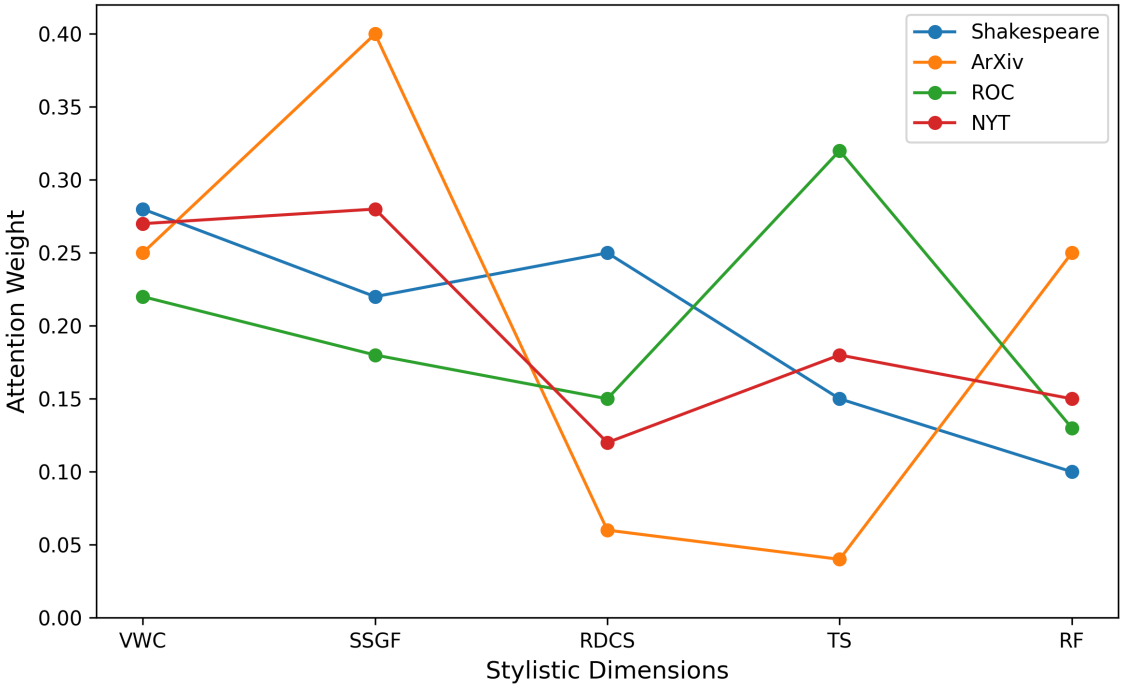


Fig. 8. Interpretability evaluation experiments results.

show high weights in VWC and RDCS, highlighting the rich vocabulary, figurative language, and poetic structures typical of early modern English drama. ArXiv papers, on the other hand, place the highest weight on SSGF, reflecting their focus on precise sentence construction and strict grammatical correctness, while the weights for rhetorical elements and tone remain low due to the formal and technical nature of scientific writing. ROCStories, composed of narrative short stories, assign the largest weights to TS, emphasizing emotional development and storytelling, with moderate attention on vocabulary and syntax. New York Times articles display a relatively balanced distribution, with slightly higher weights in VWC and SSGF, consistent with journalistic requirements for clear, accurate, and readable reporting. These distributions demonstrate that attention mechanisms can effectively capture the unique stylistic traits of different datasets, supporting both interpretability and cross-genre comparison.

3.6 Hyperparameter Evaluation Experiments

Our method involves several hyperparameters that require manual tuning. However, in the previous experiments, we only presented the optimal solution obtained after adjusting the hyperparameters. We did not show the relationship between model performance and hyperparameter changes. Therefore, in this section, we will explore the extent to which model performance is affected by hyperparameter changes.

This evaluation is necessary to provide a systematic understanding of how sensitive our method is to variations in each hyperparameter. We can identify which hyperparameters have the most significant impact on key performance metrics such as TPR, FPR, and F1 score. Such insight not only supports the reproducibility of our approach but also ensures that the selected hyperparameter settings are robust across different datasets and experimental conditions.

Moreover, this evaluation aligns with the project goals by confirming that our method maintains stable and reliable performance under a range of plausible hyperparameter configurations. It provides a principled justification for the choices made in previous experiments and helps guide future applications or extensions of our watermark detection framework.

3.6.1 Experimental Setting

In this experiment, we specifically focus on the effects of two key hyperparameters on the performance of our model. First, we examine the threshold parameter σ used in Equations (10) and (11) during the contrastive learning phrase for distinguishing positive and negative samples. Next, we analyze the effect of the length of the watermark matrix, len(M_w), in Equation (14) during the Watermark Extraction Phase. Specifically, when evaluating the impact of σ, we fix len(M_w) at 320, and when analyzing the effect of len(M_w), we set σ to the median value of 70% of the samples.

3.6.2 Experimental Results

Figures 9 and 10 illustrate the impact of two key hyperparameters—σ (contrastive learning threshold) and $len(M_w)$ (watermark matrix length)—on model performance.

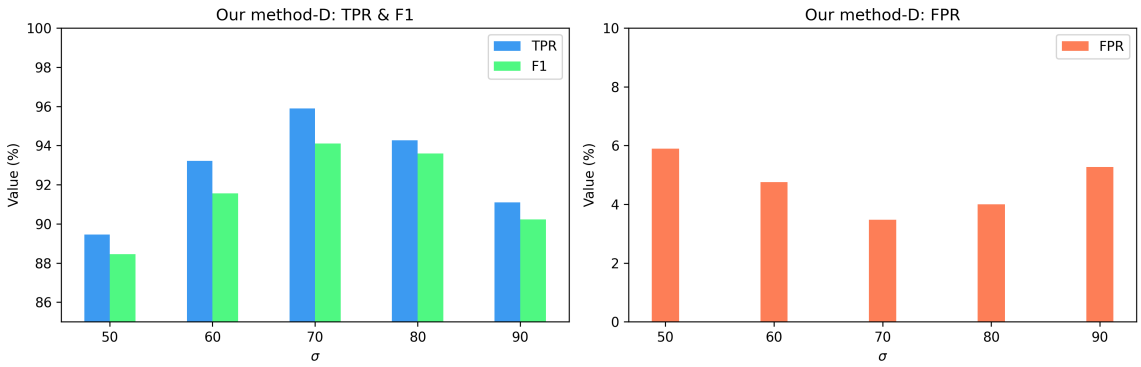


Fig. 9. Impact of the contrastive learning threshold σ on model performance.

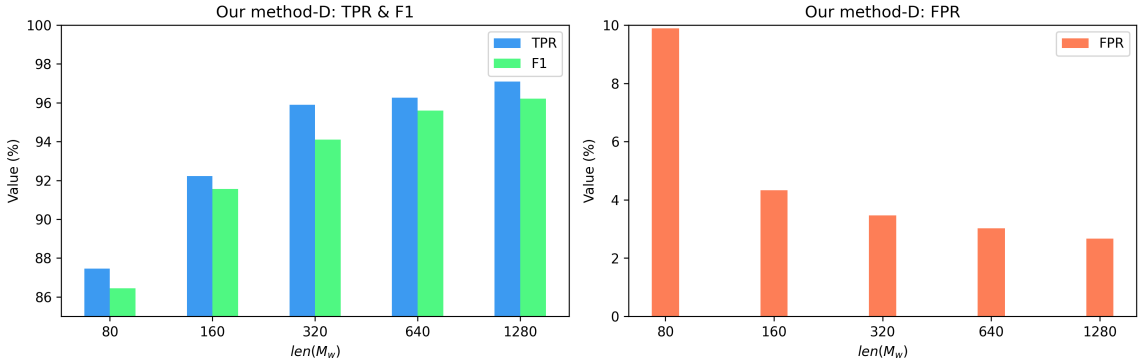


Fig. 10. Impact of the contrastive learning threshold $len(M_w)$ on model performance.

By analyzing the results shown in Figures 9, it can be observed that selecting an appropriate classification threshold is crucial. The model achieves optimal performance when σ is set to the median value corresponding to 70% of the samples. Deviating from this value in either direction leads to performance degradation. Specifically, a threshold that is too high causes the model to classify stylistic features predominantly as negative samples, whereas a threshold that is too low results in excessive classification as positive samples. Both scenarios introduce an imbalance between positive and negative samples, making it difficult for the model to effectively learn the distinctions between them, which in turn adversely affects the final performance.

By analyzing the results shown in Figures 10, we can observe a clear positive correlation: as the length of the watermark matrix increases, the model performance also improves. This is because a longer watermark matrix provides more parameters to represent the stylistic mapping of text, thereby theoretically enhancing the model's ability to capture and express textual style. However, increasing the watermark matrix length also results in higher computational costs, making it essential to balance matrix length and computational resources in practical applications. Notably, when the watermark matrix length is set to 80,

there is a sudden drop in performance. Therefore, even when computational costs are taken into account, the results indicate that overly restrictive limits on the watermark matrix length should be avoided.

4 Conclusion

4.1 Project Outcomes

In this project, we adapted a state-of-the-art digital watermarking algorithm that does not rely on the “green lists” mechanism to safeguard modern textual datasets. Through extensive experiments under diverse conditions, we demonstrated the effectiveness, robustness, and reliability of the proposed approach.

4.2 Analysis of the Project Approach

4.2.1 Complexity

The proposed method demonstrates a high degree of design complexity, as it integrates multi-stage processing and heterogeneous technical components. Specifically, the framework is divided into three main phases:

- **Comparative Learning Phase**: Sentence embeddings are extracted using an encoder. Pseudo-label generation, contrastive loss, and instance selection mechanisms are employed to construct positive and negative sample sets.
- **Condensed Lists Building Phase**: The large language model (LLM) generates explanatory sentences for five stylistic dimensions. These are encoded into a style matrix, and an attention mechanism is applied to dynamically weight and fuse features for each style dimension.
- **Watermark Extraction Phase**: A learnable watermark matrix maps the weighted style features into a binary watermark vector. Style anchors and a convergence loss function are introduced to enhance stability and robustness.

Overall, the design integrates comparative learning, deep learning (encoder-based feature extraction), LLM prompt engineering, attention mechanisms for stylistic weighting, and cryptography-inspired mapping (watermark matrix with sigmoid-based binary anchor generation), highlighting its notable design complexity.

The proposed artefact also exhibits high functional complexity, as it implements multiple advanced functionalities:

- **Multi-dimensional Style Analysis**: Explicit disentanglement and quantification of five independent style dimensions (lexical, syntactic, rhetorical, emotional, and rhythmic) using LLMs, rather than relying on traditional word frequency statistics.
- **Dynamic Feature Weighting**: An attention mechanism dynamically learns the contribution weights of each style dimension to the "protected style". This enables adaptive feature fusion and extends applicability to diverse stylistic contexts.
- **Discrete Watermark Generation**: Continuous style features are mapped into compact binary watermark vectors through a learnable matrix (M_w) and a sigmoid transformation.
- **Few-shot Capability**: The few-shot evaluation experiments indicate that the proposed framework requires only a limited number of positive samples (T_P) to construct effective watermark anchors. This finding highlights the framework's potential in few-shot learning scenarios, suggesting that reliable style-infringement detection can be achieved even under data-scarce conditions.

4.2.2 Challenges

Despite its high-complexity design and functionality, the proposed method still faces several challenges:

1. **Learning Rate Configuration**: The method involves multiple stages, each with distinct sets of parameters that require backpropagation. Determining appropriate learning rates for different stages is non-trivial, and a considerable amount of exploration is necessary to identify an optimal configuration.

2. **Training Overhead from Attention Weighting**: Although the introduction of an attention matrix improves accuracy by dynamically weighting features, it also makes the model harder to converge. This often requires additional training epochs to achieve optimal performance, thereby increasing computational overhead.

3. **Prompt Template Design**: The construction of condensed lists is highly dependent on the design of prompt templates. Moreover, results from the interpretability evaluation experiments reveal that the contribution weights of stylistic dimensions vary across different types of text. This suggests that a single universal prompt may not be sufficient, and designing adaptive or style-specific prompt templates remains an open challenge.

4. **LLM API Bottleneck**: During the Condensed Lists Building Phase, each sentence requires querying an LLM to generate stylistic explanations. As the dataset size increases, the latency and financial cost of repeated API calls become a critical bottleneck for large-scale deployment.

5. **Sensitive to Certain Hyperparameters**: The decision boundary for watermark verification depends heavily on hyperparameter settings. In particular, the threshold $\epsilon = 1\% \times len_w$ significantly affects the detection results. However, this choice lacks rigorous theoretical justification or robustness experiments, raising concerns about its generalizability. In addition, the results of hyperparameter evaluation experiments also show that the model performance is related to the contrastive learning threshold and watermark matrix length parameter settings. Therefore, finding an appropriate hyperparameter is a challenging task.

4.2.3 Scope

The scope of this project is strictly confined to the task of **LLM-generated text style infringement detection**. The core objective is to construct implicit watermark anchors for a specific human writing style. Given a suspected text, the method quantifies its stylistic imitation degree with respect to using the Hamming distance. The framework is designed as a complete end-to-end detection system. However, it does not extend to broader tasks such as generic text watermarking, content authentication, or deepfake detection.

In terms of **text granularity**, the method currently supports sentence-level detection. In

principle, paragraph- or document-level detection can be achieved by repeatedly applying sentence-level evaluations.

In terms of **text content**, the interpretability evaluation experiments demonstrate that the proposed method achieves robust watermark detection performance across diverse stylistic text types, thereby ensuring broad adaptability to different textual domains.

In summary, the scope of this project is neither overly narrow nor excessively general. It balances between focus and adaptability: focusing exclusively on style-infringement detection within LLM-generated texts while retaining flexibility across multiple stylistic dimensions and text domains.

4.2.4 Execution Quality

In our watermark detection experiments we found that our method achieves superior performance compared to all baselines. Specifically, on the DeepSeek platform (Our method-D), the best TPR reaches $95.89\% \pm 2.31\%/95.22\% \pm 2.75\%$, with F1 scores of $94.11\% \pm 3.02\%/93.87\% \pm 2.51\%$. On the Grok platform (Our method-G), the best TPR reaches $94.42\% \pm 2.63\%/91.31\% \pm 3.86\%$, with F1 scores of $93.78\% \pm 2.28\%/92.64\% \pm 3.12\%$. Furthermore, ablation studies verify the indispensability of each component in our framework, confirming their collective contribution to the final performance. Taken together, these results highlight the technical accuracy of our approach.

In terms of reliability, robustness evaluation experiments show that the model maintains strong performance under various adversarial attacks, with only a slight overall degradation. This indicates the robustness and anti-interference capability of our method. Moreover, interpretability evaluation experiments reveal that our approach is applicable across diverse text styles, offering broad adaptability. The learned attention weights further provide interpretable insights into the contribution of different style dimensions, making the method both transparent and user-friendly.

Overall, these findings indicate that the proposed framework demonstrates a high level of execution quality, successfully integrating accuracy, robustness, and interpretability into a coherent and practically viable watermarking solution.

4.3 Future Work

Despite the high execution quality demonstrated by the proposed framework, several directions remain open for further research. First, the issue of hyperparameter sensitivity warrants deeper investigation. A more principled approach to setting key parameters could be developed to replace heuristic-based choices. Second, during the condensed list construction stage, the design of prompt templates remains critical. Future work could explore automatic prompt engineering techniques to improve template efficiency. Third, to alleviate the bottleneck of LLM APIs, offline approaches may be considered to extend the framework without incurring excessive computational costs. Moreover, the current method supports style protection only at the sentence level; extending it to paragraph- or document-level detection would further enhance its applicability. Finally, the framework is currently limited to protecting a single style at a time, and future work could investigate multi-style protection mechanisms to broaden its utility.

Overall, these future directions could substantially enhance the scalability, robustness, and practical value of the framework.

References

Bergman, A Stevie et al. (2022). “Guiding the release of safer E2E conversational AI through value sensitive design”. In: *Proceedings of the 23rd Annual Meeting of the Special Interest Group on Discourse and Dialogue*. Association for Computational Linguistics.

Brown, Tom et al. (2020). “Language models are few-shot learners”. In: *Advances in Neural Information Processing Systems* 33, pp. 1877–1901.

Chang, Hongyan, Hamed Hassani, and Reza Shokri (2024). “Watermark smoothing attacks against language models”. In: *arXiv preprint arXiv:2407.14206*.

Dathathri, Sumanth et al. (2024). “Scalable watermarking for identifying large language model outputs”. In: *Nature* 634.8035, pp. 818–823.

DeepSeek (2024). *DeepSeek-V3 Technical Report*. `https://github.com/deepseek-ai/DeepSeek-V3`.

Devlin, Jacob et al. (2019). “Bert: Pre-training of deep bidirectional transformers for language understanding”. In: *Proceedings of the 2019 conference of the North American*

chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers), pp. 4171–4186.

Dugan, Liam et al. (2024). “Raid: A shared benchmark for robust evaluation of machine-generated text detectors”. In: *arXiv preprint arXiv:2405.07940*.

Gao, Tianyu, Xingcheng Yao, and Danqi Chen (2021). “Simcse: Simple contrastive learning of sentence embeddings”. In: *arXiv preprint arXiv:2104.08821*.

Holtzman, Ari, Jan Buys, Li Du, et al. (2019). “The curious case of neural text degeneration”. In: *arXiv preprint arXiv:1904.09751*.

Holtzman, Ari, Jan Buys, Maxwell Forbes, et al. (2018). “Learning to write with cooperative discriminators”. In: *arXiv preprint arXiv:1805.06087*.

kingburrito666 (2022). *Shakespeare Plays*. https://www.kaggle.com/datasets/kingburrito666/shakespeare-plays. Accessed: 2025-08-13.

Kirchenbauer, John et al. (2023). “A watermark for large language models”. In: *International Conference on Machine Learning*. PMLR, pp. 17061–17084.

Loshchilov, Ilya and Frank Hutter (2017). “Decoupled weight decay regularization”. In: *arXiv preprint arXiv:1711.05101*.

Lu, Yijian et al. (2024a). “An entropy-based text watermarking detection method”. In: *arXiv preprint arXiv:2403.13485*.

— (2024b). “An entropy-based text watermarking detection method”. In: *arXiv preprint arXiv:2403.1348*

Masons, Pinsent (2024). “Getty Images v Stability AI: the implications for UK copyright law and licensing”. In: *Pinsent Masons*. Accessed: 2025-08-07. URL: https://www.pinsentmasons.com/out-law/analysis/getty-images-v-stability-ai-implications-copyright-law-licensing.

Megías, David et al. (2021). “Dissimilar: Towards fake news detection using information hiding, signal processing and machine learning”. In: *Proceedings of the 16th International Conference on Availability, Reliability and Security*, pp. 1–9.

Michael Grynbaum, Ryan Mac (2023). “The Times Sues OpenAI and Microsoft Over A.I. Use of Copyrighted Work”. In: *The New York Times*. Accessed: 2025-08-07. URL: https://www.nytimes.com/2023/12/27/business/media/new-york-times-open-ai-microsoft-lawsuit.html.

Milmo, Dan (2023). “Sarah Silverman sues OpenAI and Meta claiming AI training infringed copyright”. In: *The Guardian*. Accessed: 2025-08-07. URL: https://www.theguardian.com/technology/2023/jul/10/sarah-silverman-sues-openai-meta-copyright-infringement.

Mirsky, Yisroel et al. (2023). “The threat of offensive ai to organizations”. In: *Computers & Security* 124, p. 103006.

OpenAI (2025). *Introducing GPT-5*. https://openai.com/zh-Hans-CN/index/introducing-gpt-5/. Accessed: 2025-08-15.

Pan, Leyi et al. (2024). “Markllm: An open-source toolkit for llm watermarking”. In: *arXiv preprint arXiv:2405.10051*.

Rastogi, Saksham and Danish Pruthi (2024). “Revisiting the robustness of watermarking to paraphrasing attacks”. In: *arXiv preprint arXiv:2411.05277*.

Sahoo, Pranab et al. (2024). “A systematic survey of prompt engineering in large language models: Techniques and applications”. In: *arXiv preprint arXiv:2402.07927*.

Schatz, Casca (2022). *New York Times Articles Metadata 2020-2022*. https://www.kaggle.com/datasets/cascaschatz/new-york-times-articles-metadata-2020-2022/data. Accessed: 2025-08-11.

submitters, arXiv.org (2024). *arXiv Dataset*. DOI: 10.34740/KAGGLE/DSV/7548853. URL: https://www.kaggle.com/dsv/7548853.

XAI (2024). *Introducing Grok: The AI Assistant Built by xAI*. https://x.ai/blog/grok.

Zhang, Ziwei et al. (2025). “MiZero: The Shadowy Defender Against Text Style Infringements”. In: *arXiv preprint arXiv:2504.00035*.

Zhao, Xuandong et al. (2023). “Provable robust watermarking for ai-generated text”. In: *arXiv preprint arXiv:2306.17439*.

Zhu, Xuekai et al. (2022). “StoryTrans: Non-parallel story author-style transfer with discourse representations and content enhancing”. In: *arXiv preprint arXiv:2208.13423*.

Appendices

A Prompt Templates

The following Figure 11 shows the prompt word template used to build condensed lists:

```
(task discription)
You are an excellent linguist in the domain of text style. <Sentence 2> is known to have the
same text style as <Sentence 1>. Your task is to extract similarities in the textual style of
<Sentence 1> and <Sentence 2> based on the following five aspects.
```

```
(analysis)
- **Vocabulary and Word Choice**: Consider whether the two sentences use similar
vocabulary or use a specific type of language related to the topic, write what they have in
common, e.g., Old English, Internet slang, etc.
- **Syntactic Structure and Grammatical Features**: Look for similarities in sentence
structure specific to the topic, like technical terminology or specialized grammar.
- **Rhetorical Devices and Stylistic Choices**: Identify the use of rhetorical devices
specific to the topic, such as scientific metaphors, historical allusions, etc.
- **Tone and Sentiment**: Compare the tone and sentiment in both sentences within the
context of the topic being discussed, Such as narcissism, pessimism, cynicism, etc.
- **Rhythm and Flow**: Evaluate the rhythm and flow of the sentences in relation to the
topic, considering any stylistic choices related to the topic's nature.
```

```
(fixed output formats)
Ensure each aspect is elaborated with a detailed sentence that captures the essence of the
feature without introducing additional text, explanations, or line breaks. Output each
description as part of the style feature list using the specified format:
`aspect1=[detailed_sentence1], aspect2=[detailed_sentence2], aspect3=[detailed_sentence3],
aspect4=[detailed_sentence4], aspect5=[detailed_sentence5]`Do not include any explanations,
or line breaks. Ensure the output is a single line and follows
the exact syntax.
```

(a)

```
(task discription)
You are an excellent linguist in the domain of text style. Your task is to extract the following
five style aspects in the <Sentence>.
```

```
(analysis)
- **Vocabulary and Word Choice**: Specify words or language choices.
- **Syntactic Structure and Grammatical Features**: Point out the sentence structure or
grammar.
- **Rhetorical Devices and Stylistic Choices**: Highlight rhetorical devices or stylistic
elements.
- **Tone and Sentiment**: Describe tone and emotional content that distinguish.
- **Rhythm and Flow**: Discuss rhythm, pacing, or flow.
```

```
(fixed output formats)
Ensure each aspect is elaborated with a detailed sentence that captures the essence of the
feature without introducing additional text, explanations, or line breaks. Output each
description as part of the style feature list using the specified format:
`aspect1=[detailed_sentence1], aspect2=[detailed_sentence2], aspect3=[detailed_sentence3],
aspect4=[detailed_sentence4], aspect5=[detailed_sentence5]`Do not include any explanations, or
line breaks. Ensure the output is a single line and follows
the exact syntax.
```

(b)

Fig. 11. Prompts for building condensed lists. (a) Details of q_p (b) Details of q_n

B Details of Experimental Setting

We use MarkLLM (Pan et al. 2024), an open-source toolkit for LLM Watermarking, available on GitHub to implement the state-of-the-art watermarking methods in this project. The experimental setup for the state-of-the-art watermarking methods is as follows: For baseline watermarking methods, the green list ratio is fixed at 0.5. The total number of green tokens in a text is approximated by a normal distribution with variance $\delta^2 = 1.5$, and a z-score threshold of 5.0 is applied.

```
(Shakespeare)
No, I'll give thee thy due, thou hast paid all there.
He durst as well have met the devil alone.
That wished him on the barren mountains starve.

(ArXiv)
We give a quantitative analysis of clustering in a stochastic model of\none-dimensional gas.
We review recent progress in operator algebraic approach to conformal quantum\nfield theory.
Most recently, both BaBar and Belle experiments found evidences of neutral\n$D$ mixing.

(ROCStories)
Marcus was happy to have the right clothes for the event.
The technician called her name and she paid for the prescription.
A recent study by Fred was glad that his feet no longer got hot in the summer.

(NYT)
When New York City was hit with a snow squall a few weeks ago, Albert Hei, 29, took refuge in an igloo.
A brand-new cottage in Oxford, Miss.; 1967 modern-style home in Kansas City, Mo.; and a three-bedroom house with red-rock views in Sedona, Ariz.
A recent study by Vacasa, a property management company, crunched data on about 500,000 American rentals.
```

Fig. 12. Example sentences from all datasets

C Example Sentences of All Datasets

The following Figure 12 shows example sentences from all the datasets used in this project. We randomly selected three example sentences from each dataset. It is worth mentioning that for the arxiv dataset, we only used the paper abstract data.